Smart Weather Alert System

Step-by-Step Development Process

We will build the Smart Weather Alert System using VS Code, HTML, CSS, JavaScript, Node.js, Express.js and a Weather API. The application will allow users to enter a city, check live weather conditions, and receive automatic alerts when dangerous weather conditions occur.

1. Project Overview

Project name: Smart Weather Alert System

Main objective: To monitor weather conditions and automatically notify users about dangerous weather.

Main features

1. Search weather by city name.
2. Detect user's location (optional).
3. Display live temperature and weather conditions.
4. Show humidity, wind speed and rainfall.
5. Provide weather forecasts.
6. Send email alerts automatically.
7. Allow users to customize alert settings.
8. Detect heavy rain, storms, high temperatures and strong winds.
9. Store user preferences in MongoDB.
10. Provide a responsive dashboard.

2. Technologies Required

Technology	Purpose
HTML	Webpage structure
CSS	Website design
JavaScript	Frontend functionality
Node.js	Backend
Express.js	Server and API
OpenWeather API	Weather information
MongoDB	Store user details
Nodemailer	Send email alerts
node-cron	Automatic weather checking

3. Install the Required Software

- **Visual Studio Code:** https://code.visualstudio.com/
- **Node.js:** https://nodejs.org/
- **MongoDB:** https://www.mongodb.com/try/download/community

After installation, open VS Code and create a project folder named: `smart-weather-alert-system`

4. Create the Project Structure

Create the following folders and files:

```
smart-weather-alert-system/
|
|-- config/
|   `-- db.js
|
|-- controllers/
|   `-- weatherController.js
|
|-- models/
|   `-- User.js
|
|-- routes/
|   `-- weatherRoutes.js
|
|-- services/
|   |-- weatherService.js
|   |-- alertService.js
|   `-- emailService.js
|
|-- public/
|   |-- index.html
|   |-- style.css
|   `-- script.js
|
|-- .env
|-- .gitignore
|-- package.json
`-- server.js
```

This structure separates frontend, backend, database and weather alert functionality.

5. Initialize the Node.js Project

Open the VS Code terminal and run:

```
npm init -y
```

Install the required packages:

```
npm install express axios mongoose dotenv nodemailer node-cron cors
```

Install nodemon for development:

```
npm install --save-dev nodemon
```

6. Get Your Weather API Key

We will use the OpenWeather service to retrieve live weather information. Documentation: https://openweathermap.org/api

Steps

1. Open the OpenWeather website.
2. Create an account.
3. Navigate to API keys.
4. Generate an API key.
5. Copy the key.
6. Add it to your `.env` file.

Example:

```
PORT=5000
WEATHER_API_KEY=your_api_key
MONGO_URI=your_mongodb_url
EMAIL_USER=your_email
```

```
EMAIL_PASS=your_email_app_password
```

Never upload your .env file publicly.

7. Create the Backend Server

Open `server.js`. This file starts your Express server and connects all backend routes.

Basic code:

```
const express = require("express");
const dotenv = require("dotenv");
const cors = require("cors");

dotenv.config();

const app = express();

app.use(cors());
app.use(express.json());
app.use(express.static("public"));

app.get("/", (req, res) => {
    res.sendFile(__dirname + "/public/index.html");
});

app.listen(process.env.PORT || 5000, () => {
    console.log("Weather server running");
});
```

Run the server:

```
node server.js
```

Open your browser: http://localhost:5000

8. Connect the Weather API

Create `services/weatherService.js`. The backend will:

1. Receive the city name.
2. Request weather information from OpenWeather.
3. Receive the JSON response.
4. Extract temperature, humidity and wind speed.
5. Send the data to the frontend.

Example API request:

```
const axios = require("axios");

async function getWeather(city) {
    const url = "https://api.openweathermap.org/data/2.5/weather";

    const response = await axios.get(url, {
        params: {
            q: city,
            appid: process.env.WEATHER_API_KEY,
            units: "metric"
        }
    });

    return response.data;
}

module.exports = getWeather;
```

The API supports temperature, humidity, wind speed, rainfall and weather descriptions (source: openweathermap.org).

9. Create Weather Alert Conditions

Create `services/alertService.js`. This is the most important feature of your project. Set example alert conditions:

Weather condition	Example threshold
High temperature	Above 40°C
Strong wind	Above 50 km/h
Heavy rain	Above 10 mm/hour
Storm	Thunderstorm detected

Example code:

```
function checkAlerts(weather) {
    let alerts = [];

    if (weather.main.temp > 40) {
        alerts.push("High Temperature Alert!");
    }

    if (weather.wind.speed * 3.6 > 50) {
        alerts.push("Strong Wind Alert!");
    }

    if (weather.rain && weather.rain["1h"] > 10) {
        alerts.push("Heavy Rain Alert!");
    }

    if (weather.weather[0].main === "Thunderstorm") {
        alerts.push("Storm Alert!");
    }

    return alerts;
}

module.exports = checkAlerts;
```

Note: These are example project thresholds, not official severe-weather warning standards. For real safety warnings, use authoritative local meteorological alerts.

10. Create Email Notification System

Create `services/emailService.js`. Use Nodemailer to send email notifications. Your application should:

1. Receive the user's email address.
2. Check weather conditions.
3. Identify dangerous conditions.
4. Generate an alert message.
5. Send the notification by email.

Example:

```
const nodemailer = require("nodemailer");

async function sendAlert(email, message) {
    const transporter = nodemailer.createTransport({
        service: "gmail",
        auth: {
            user: process.env.EMAIL_USER,
            pass: process.env.EMAIL_PASS
        }
    });

    await transporter.sendMail({
```

```
        from: process.env.EMAIL_USER,
        to: email,
        subject: "Weather Alert!",
        text: message
    });
}

module.exports = sendAlert;
```

Use a Gmail App Password where applicable, not your normal Gmail password.

11. Automate Weather Checking

Create an automatic checking system using `node-cron`. For example:

- Every 15 minutes, retrieve weather information.
- Check alert conditions.
- Compare them with user preferences.
- Send an email if a new alert is detected.

Basic scheduling example:

```
const cron = require("node-cron");

cron.schedule("*/15 * * * *", () => {
    console.log("Checking weather conditions...");
});
```

Important: This only schedules the task. You must connect it to your weather-checking and email functions. Also, prevent duplicate emails for the same ongoing alert.

12. Create the Frontend Dashboard

Now create your website using three files: `public/index.html`, `public/style.css` and `public/script.js`. Your dashboard should contain:

A. Search section

- City name input
- Search button
- Current location button

B. Weather information

- Temperature
- Weather condition
- Humidity
- Wind speed
- Rainfall

C. Alert settings

- Email address
- Rain alert checkbox
- Temperature alert checkbox
- Wind alert checkbox
- Storm alert checkbox

D. Notification section

- Active alerts

- Alert history
- Date and time
- Warning messages

13. Connect Frontend and Backend

In `public/script.js`, use JavaScript `fetch()` to communicate with the backend. Example:

```
async function searchWeather() {
    const city = document.getElementById("city").value;

    const response = await fetch(
        `/api/weather/${encodeURIComponent(city)}`
    );

    const data = await response.json();

    console.log(data);
}
```

You must create the corresponding Express route in `routes/weatherRoutes.js` and connect it to `server.js`.

14. Connect MongoDB

Create `config/db.js`. MongoDB will store:

- User email addresses
- Selected cities
- Alert preferences
- Temperature thresholds
- Alert history

Create a `User.js` model to define the database structure. You can use MongoDB Atlas if you don't want to install MongoDB locally: https://www.mongodb.com/products/platform/atlas-database

15. Test the Complete Project

Test each feature separately before testing everything together.

Test	Expected result
Search Vijayawada	Weather information appears
Search Hyderabad	Weather information appears
Temperature above threshold	Temperature alert
Heavy rainfall condition	Rain alert
Strong wind condition	Wind alert
Storm condition	Storm alert
Enter valid email	Alert email delivered
Disable rain alert	No rain email sent
Automatic checking	Weather checked on schedule

16. Run the Complete Project

Open the VS Code terminal and run:

```
npm run dev
```

Open: http://localhost:5000

Your complete application should now have a working weather dashboard, weather API connection, alert detection, email notifications and automatic weather checking.

17. Final Project Workflow

This is how all the modules work together:

1	User enters city and email (frontend dashboard)
2	Node.js + Express backend
3	Weather API retrieves live data
4	Weather alert detection
5	MongoDB saves user settings
6	Email alert notifies the user

18. Recommended Development Order and Checklist

Don't develop everything at once. Follow this order:

[ ] **Step 1:** Install VS Code and Node.js

[ ] **Step 2:** Create project folder and files

[ ] **Step 3:** Initialize Node.js and install packages

[ ] **Step 4:** Get OpenWeather API key

[ ] **Step 5:** Create Express backend

[ ] **Step 6:** Connect Weather API

[ ] **Step 7:** Build weather dashboard

[ ] **Step 8:** Implement alert detection

[ ] **Step 9:** Implement email notifications

[ ] **Step 10:** Configure automatic weather checking

[ ] **Step 11:** Connect MongoDB

[ ] **Step 12:** Test all features

[ ] **Step 13:** Deploy the application

Recommendation: First complete the live weather dashboard, then add automatic alerts and email notifications. This makes debugging easier.

Important: Use official weather warnings from the relevant meteorological authority, such as the India Meteorological Department, alongside forecast-based alerts. Do not treat your sample thresholds as official emergency warnings.